

Iptables

Cybersecurity Internship – Phase 1, Task 10

Name: Parmar Rakesh

Date: 26, AUG, 2025

Internship Program: Cybersecurity Internship – UJAR TECH

What are Firewalls and Iptables?

A firewall serves as a security system that governs incoming and outgoing network traffic based on predetermined security rules. Essentially, it acts as a barrier between your trusted internal network and untrusted external networks. Linux, being a popular choice for servers and network devices, uses iptables as its firewall management tool, enabling users to create and manage packet filtering rules.

Iptables works by filtering packets in the Linux kernel, allowing users to define rules that determine how packets are handled. These rules can specify whether to accept, drop, or redirect packets based on criteria such as source IP address, destination IP address, protocol, and port number.

Installing Iptables

Most Linux distributions come with iptables pre-installed. You can check if iptables is present on your system by running:

- **iptables --version**

If you do not have iptables installed, you can typically install it using your distribution's package manager. For example:

On Debian/Ubuntu:

- **sudo apt update**
- **sudo apt install iptables**

Architecture of Iptables

Iptables operates using a set of tables and chains. The primary tables used are:

- **filter table:** The default table used for packet filtering. It includes three default chains: INPUT (for incoming connections), OUTPUT (for outgoing connections), and FORWARD (for forwarded connections).
- **nat table:** Used for Network Address Translation purposes, especially relevant for setting up port forwarding.
- **mangle table:** Used for specialized packet alterations, such as adjusting the Quality of Service (QoS) of packets.

Chains and Rules

Chains are lists of rules that are applied to packets as they flow through the firewall. Each rule specifies a specific condition and a target action (ACCEPT, DROP, REJECT, etc.) based on the condition match.

Understanding Iptables Commands

The basic syntax for iptables commands is:

- **iptables [options] [CHAIN] [RULES]**

Here's what each part means:

- options: Command flags (e.g., **-A** to append, **-D** to delete).
- **CHAIN**: The chain where the rule will be applied (INPUT, OUTPUT, FORWARD).
- **RULES**: The actual rules defining conditions and targets.

Some commonly used options include:

- **-A**: Append a rule to a chain.
- **-D**: Delete a rule from a chain.
- **-L**: List rules in a chain.
- **-F**: Flush all rules in a chain.
- **-P**: Set the default policy for a chain.

Now that we understand the basic building blocks of iptables, let's explore how to use it for basic firewall configurations.

Basic Firewall Configuration with Iptables

1. Setting Default Policies

Before adding rules, it's best practice to define default policies for each chain in the **filter** table. This way, you explicitly define how to handle packets that do not match any rule. Set the default policy to DROP (deny all traffic) for the INPUT, OUTPUT, and FORWARD chains:

- **sudo iptables -P INPUT DROP**
- **sudo iptables -P OUTPUT ACCEPT**
- **sudo iptables -P FORWARD DROP**

2. Allowing Established Connections

To keep established connections alive while dropping new and unrequested packets, you can allow traffic related to already established connections:

- **sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT**

3. Allowing SSH Access

To allow SSH (Secure Shell) access to your server on the default port (22), you can add the following rule:

- **sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT**

4. Allowing HTTP and HTTPS Traffic

For web servers, allow HTTP (port 80) and HTTPS (port 443) traffic:

- **sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT**

- **sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT**

5. Allowing ICMP (Ping) Requests

Allowing ping requests helps in network troubleshooting:

- **sudo iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT**

6. Saving Your Rules

To ensure that your iptables rules persist across reboots, you'll need to save them:

- **sudo iptables-save > /etc/iptables/rules.v4**

7. Port Forwarding

Port forwarding allows traffic on a specific port to be redirected to a different IP address or port. This is particularly useful for running services behind a NAT (Network Address Translation).

Assuming you want to forward traffic from port 80 (HTTP) to an internal server with IP

192.168.1.100, use:

- **sudo iptables -t nat -A PREROUTING -p tcp --dport 80 -j DNAT --to-destination 192.168.1.100:80**
- **sudo iptables -A FORWARD -p tcp -d 192.168.1.100 --dport 80 -j ACCEPT**

8. Rate Limiting

Rate limiting helps to mitigate DoS (Denial of Service) attacks by limiting the amount of requests a single IP can make to your server. For example, to limit incoming SSH connections to 5 requests per minute:

- **sudo iptables -A INPUT -p tcp --dport 22 -i eth0 -m state --state NEW -m recent --set**
- **sudo iptables -A INPUT -p tcp --dport 22 -i eth0 -m state --state NEW -m recent --update --seconds 60 --hitcount 5 -j DROP**

9. Logging with Iptables

Logging traffic is critical for monitoring and troubleshooting. You can add logging rules to view detailed information about packets being dropped or accepted.

For example, to log dropped packets, use:

- **sudo iptables -A INPUT -j LOG --log-prefix "Dropped Packet: " --log-level 4**

To view log entries instantly, you can check the system logs:

- **sudo tail -f /var/log/syslog**

